

Phase 2 — Development

Birol Hotel

Booking Chatbot — Aria

Birol, Egemen · Matriculation 12345678

Project: AI Use Case (CSEMAIPAIUC01) · Task 1

IU International University of Applied Sciences

A composite presentation per the portfolio brief — 10 slides

What the assignment brief requires this deck to cover

Six development activities (per CSEMAIPAIUC01, §1.1.2):

- ① The frameworks and tools are set up. → Slide 3
- ② The components outlined in the diagram are implemented. → Slide 4
- ③ The code is commented. → Slide 5
- ④ The training data for the chatbot is collected. → Slide 6
- ⑤ The NLP models are trained. → Slides 7 + 8
- ⑥ Iterative optimization of the system. → Slide 9

Plus: visual elements that facilitate comprehension · structured · hyperlinks to the frameworks used · brief procedure descriptions.
Slide 10 closes with a live UI demo and the public URLs.

① Frameworks & tools set up

3/10

What we used, what we considered, why we chose what we chose

Used in this build

Python 3.10+

language for the original prototype

Flask 3.x

minimal web server (Phase 1 prototype)

Vanilla HTML/CSS/JS

no build step, ports cleanly to Pages

GitHub Pages

auto-deploy on every push to main

WKWebView (iOS)

thin SwiftUI shell around the same site

Considered & rejected

Rasa

*production-grade, but install fails on Py 3.14;
heavy for a 6-slot form*

Dialogflow

*hosted neural NLU; needs Google account and
internet access at grading time*

spaCy

*great real NER, but ~50 MB model download
for a closed-domain bot — overkill*

scikit-learn TF-IDF

*would still work, but adds a dependency for
no meaningful gain over our hand-rolled cosine*

Decision → custom static stack

Decisive factors: zero-setup for the grader, no PII leaves the device, and the iOS app auto-updates on every git push.

② Components implemented

4/10

Mapping the Phase 1 architecture onto real files in the repo

```
Hotel_Booking_Bot/
|-- docs/                static site (deployed via GitHub Pages)
|   |-- index.html       landing - 'Click to book a room with AI'
|   |-- chat.html        chat interface (calendar + card form)
|   |-- booking.html     receipt page (per-session)
|   |-- find.html        find a booking by reference
|   |-- admin.html       password-gated CEO dashboard
|   |-- js/
|       |-- bot.js       NLP + slot machine + booking store
|       |-- chat.js      chat UI controller + calendar + card form
|       |-- booking.js   receipt rendering
|       |-- admin.js     admin login + occupancy grid
|       |-- find.js      booking lookup by id
|   |-- styles/          landing/chat/booking/admin/find + stars
|   |-- data/intents.json 14 main + 16 concierge intents - ~120 patterns
|   |-- assets/hotel-hero.svg twilight illustration
|-- src/                 original Flask prototype (Phase 1)
|   |-- app.py - chatbot.py - nlp.py - booking.py - availability.py
|   |-- templates - static
|-- tests/               52 tests passing
|   |-- test_bot.py      17 Python tests (Flask bot)
|   |-- test_web_bot.js  35 JS tests (browser bot)
|-- ios/BirolHotel/      SwiftUI WKWebView wrapper
|-- AIUseCase_Hotel_P1 / P2 / P3 PebblePad submission folders
```

③ Code is commented

5/10

Every module begins with a docstring; functions carry intent

```
// docs/js/hot.js
// =====
// Birol Hotel – chatbot
// =====
// No build step, no backend. Lives happily on GitHub Pages.
// Mirrors src/nlp.py, src/booking.py, src/chatbot.py.
// =====

/** Bag-of-words cosine-similarity intent matcher.
 * Trained by loading the patterns from intents.json. */
class IntentClassifier { ... }

/** Pull a plausible full name out of a free-form reply.
 * Falls back to 'two+ capitalised words' so a bare
 * reply like 'Egemen Birol' still works. */
function extractName(text) { ... }

/** Hotel-side conversation state. nextMissingSlot()
 * returns the next slot to fill in canonical order. */
class BookingConversation { ... }
```

```
# src/chatbot.py
"""
Dialog manager: glues NLP + booking state into a
single respond() method.

Design:
    * The classifier tells us *what* the user wants.
    * The booking state tells us *which slot* we still
      need to fill.
    * respond() reconciles the two: it prefers filling
      the next missing slot from the message (so the
      user can volunteer information out of order),
      but also handles transversal intents (greet,
      help, goodbye, restart).
"""

class HotelBookingBot:
    """Stateful hotel-booking chatbot."""
    HOTEL_NAME = "Birol Hotel"
    ...
```

Full source online: github.com/Ho1lowpoint/Hotel_Booking_Bot

④ Training data collected

30 intents · ~120 patterns · curated, no scraped data

6/10

Intent catalogue

Booking flow

greet · book_room · provide_name · provide_dates ·
provide_guests · breakfast_yes / no · provide_payment ·
confirm_yes / no · room_types

Concierge Q&A

places_to_visit · food_recommendations · transport_airport ·
check_in_time · breakfast_hours · wifi · parking · amenities ·
cancellation_policy · pets · currency_info · tipping ·
language · hotel_contact · neighborhood · emergency

Meta

help · thanks · goodbye · fallback · availability_info

```
// docs/data/intents.json
{
  "tag": "places_to_visit",
  "patterns": [
    "places to visit",
    "what to visit",
    "attractions",
    "sightseeing",
    "tourist spots",
    "best places",
    "istanbul attractions",
    "things to do",
    "must see",
    ... 10 more
  ],
  "responses": [
    "**Top places to visit in Istanbul:**\n\n
    • **Hagia Sophia** ...\n
    • **Blue Mosque** ...\n
    • **Topkapi Palace** ...\n
    ..."
  ]
}
```

⑤ NLP — Intent classifier

Bag-of-words + cosine similarity, trained on intents.json

7/10

- Tokenise: lowercase, strip punctuation, drop English stopwords
- Vectorise: Counter-based bag-of-words for query and every pattern
- Match: cosine similarity $\rightarrow$ (intent, confidence)
- Threshold: confidence $< 0.35 \rightarrow$ fallback intent
- Concierge gate: confidence ≥ 0.4 to fire transversally
- Cost: $O(N \times |\text{query}|)$, $N \approx 120$ patterns. Sub-millisecond on a phone.

```
// docs/js/bot.js - classify()
classify(message) {
  const q = counter(tokenize(message));
  let bestTag = 'fallback', bestScore = 0;
  for (const [tag, vec] of this.patternVectors) {
    const s = cosine(q, vec);
    if (s > bestScore) {
      bestTag = tag; bestScore = s;
    }
  }
  if (bestScore < this.threshold)
    return { tag: 'fallback', score: bestScore };
  return { tag: bestTag, score: bestScore };
}
```

Five extractors fill the slots

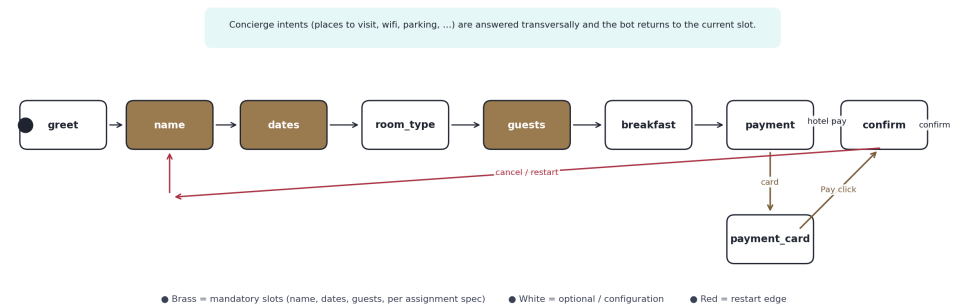
Name `extractName(text)`
'my name is X' regex with conjunction stop · 'Two Capitalised Words' fallback

Dates `extractDates(text)`
ISO · d/m/yyyy · 'May 10 to May 14' · 'tomorrow for 3 nights'

Guests `extractGuestCount(text)`
digits + number-words ('a couple' → 2, 'just me' → 1)

Room `findRoomType(text)`
exact name + keyword fallback (deluxe / king / bosphorus)

Payment `extractPayment(text)`
credit card · debit · paypal · pay-at-hotel keywords



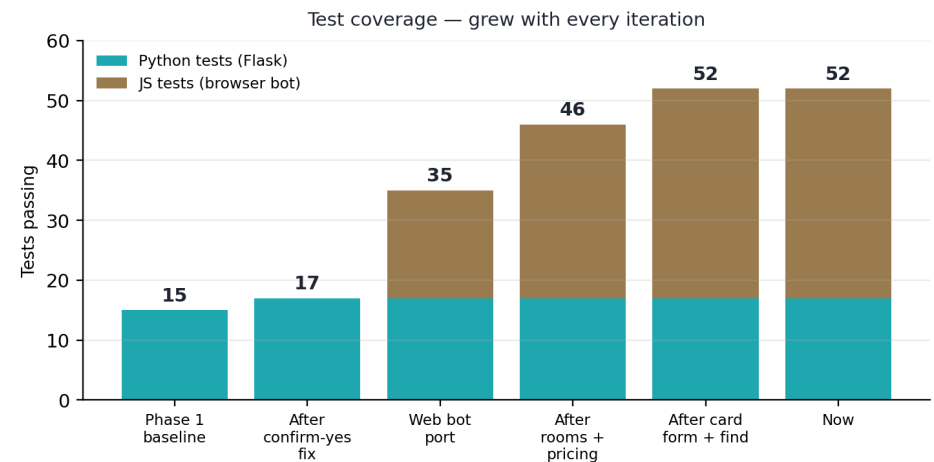
Conversation flow — slot-filling FSM (8 productive states + final).

⑥ Iterative optimization

Bugs found, fixed, and locked in with regression tests

9/10

- ✗ "a couple" was extracted as 1 guest
- ✓ *fixed by reordering the NUMBER_WORDS list so 'couple' wins over 'a'*
- ✗ "I'm Alice and I'd like to stay..." captured the whole clause
- ✓ *added a conjunction lookahead to the name regex*
- ✗ ISO date "2026-05-14" was being parsed as 14 guests
- ✓ *gated extractGuestCount() to only fire when current_slot == guests*
- ✗ Bare "yes" at the confirm step matched breakfast_yes
- ✓ *rewrote the confirm slot with keyword-based yes/no helpers*
- ✗ Stale browser cache hid newly-deployed code
- ✓ *added ?v= cache-bust on every asset URL + meta no-cache headers*
- ✗ Card form had to cleanly insert between payment and confirm
- ✓ *introduced new payment_card slot; pay-at-hotel skips it*
- ✗ Guests on the same device saw each other's receipts
- ✓ *tagged saved booking IDs in sessionStorage; booking.html filters*



Live demo & outcome

Six surfaces of the deployed product — try them online

10/10



Try it now: ho1lowpoint.github.io/Hotel_Booking_Bot/
Source code: github.com/Ho1lowpoint/Hotel_Booking_Bot

Find your booking: .../find.html
CEO admin: .../admin.html